LLM-as-a-Judge

An Evaluation Pipeline for Helpfulness, Correctness and Tone

Complete project explanation in simple language

NLP Course Project • IIIT Guwahati • 2026

What is inside: 1. The project in one minute • 2. Background topics • 3. The maths we use • 4. How our pipeline works • 5. The datasets • 6. Code walk-through • 7. How to run it • 8. Our results and what they mean • 9. Submission checklist • 10. Presentation plan • 11. Likely viva questions • 12. Glossary

1. The project in one minute

Chatbots like ChatGPT write answers in free text. How do we know whether an answer is **good**? Asking humans to read every answer is slow and expensive. So we ask **another AI** (an LLM) to act as a **judge**, like a teacher grading answer sheets.

Our judge gives every answer three marks out of 5:

Criterion	The question it asks	Example of a bad answer
Helpfulness	Does the answer actually solve the user's problem?	“Use a loop or something.”
Correctness	Are the facts and steps true?	“The capital of Australia is Sydney.”
Tone	Is it polite, respectful and suitable?	“Seriously? Look it up yourself.”

But can we **trust** the AI judge? To find out, we compare its marks with **human** marks on two datasets:

- **HelpSteer2** (120 answers): a public NVIDIA dataset where trained human annotators already rated each answer for helpfulness and correctness. Big and real.
- **Our own set** (30 answers): written by us and rated by our team members. Needed because HelpSteer2 has no **tone** rating.

If the judge's marks match the human marks well, the judge can replace humans for that criterion. This checking step is called **validation against human ratings**.

> **One-line summary for the viva:** “We built a pipeline where an LLM scores answers for helpfulness, correctness and tone using a rubric, and we validated it against real human ratings: 120 HelpSteer2 samples rated by trained annotators, plus 30 samples of our own rated by our team for tone.”

2. Background topics (explained simply)

2.1 Natural Language Processing (NLP)

NLP is the part of AI that helps computers understand and produce human language: translation, chatbots, spam filters, summarising, and so on. Our project sits in the area of **NLP evaluation**: measuring how good a language system's output is.

2.2 Language models and LLMs

A **language model** predicts the next word given the previous words. A **Large Language Model (LLM)** is a very big language model (billions of parameters) trained on a huge amount of text, built on the **Transformer** architecture. Because it has read so much, it can answer questions, write code and also **judge** text. Examples: GPT, Llama, Gemini, Claude. As our judge we use **gpt-oss-120b**, an open-weight model released by OpenAI with 120 billion parameters, which we call for free through Groq. It is a *reasoning* model: it “thinks” briefly before answering. We set its reasoning effort to **low** to keep it fast and inside the free limits.

2.3 Prompt and prompt engineering

A **prompt** is the instruction text we send to an LLM. **Prompt engineering** means writing it carefully so the model does what we want. Our judge prompt contains: the role (“You are a strict but fair evaluator”), the rubric, the question and answer, and the exact output format.

2.4 Rubric

A rubric is a marking scheme, the same one teachers use. For each criterion we describe what a 1, a 3 and a 5 look like. Giving the same rubric to the LLM and to the humans makes the comparison fair.

2.5 Likert scale (1 to 5)

A 1-5 rating scale where the numbers have an order (5 is better than 4) but are not exact measurements. Because of this ordering, we use **Spearman** correlation and **weighted kappa**, which respect order.

2.6 Temperature

Temperature controls randomness. High temperature = creative, different answer each time. **Temperature 0** = the model picks its most likely answer, so running again gives (almost) the same scores. A judge must be consistent, so we use 0.

2.7 API and "OpenAI-compatible"

An **API** lets our Python code send a prompt to an LLM running on a server and get the reply back. Many providers (Groq, Ollama, Gemini, OpenAI) accept the same request format, called **OpenAI-compatible**. So by changing just two lines in `config.py` we can switch judge models.

2.8 JSON output

JSON is a simple text format for data, e.g. `{"helpfulness": 4, "correctness": 5, "tone": 3}`. We ask the judge to reply only in JSON so our code can read the numbers automatically.

2.9 Why is evaluating LLM answers hard?

Older metrics like **BLEU** and **ROUGE** count how many words an answer shares with a "reference" answer. But two answers can use different words and both be right, or share many words and one be wrong. They also cannot measure politeness. Human evaluation is the best, but slow and costly. LLM-as-a-Judge sits in the middle: **fast and cheap like a metric, but understands meaning like a human**.

2.10 LLM-as-a-Judge

Using an LLM to grade the output of another model. Two common styles:

- **Pointwise (single-answer grading):** give one answer a score, e.g. 4/5. *This is what we do.*
- **Pairwise comparison:** show two answers and ask which is better (used in Chatbot Arena / MT-Bench).

Our judge is **reference-free**: it does not need a "correct answer key". It uses its own knowledge.

2.11 Known weaknesses (biases) of LLM judges

- **Verbosity bias:** likes longer answers even when they are not better. (Our prompt says "do not reward length".)
- **Position bias:** in pairwise mode, prefers the first (or second) answer shown.
- **Self-preference bias:** likes answers written by itself or similar models.
- **Leniency:** gives high marks too easily. (We check this with the *bias* number.)
- **Knowledge limits:** if the judge does not know a fact, it cannot judge correctness well.

These weaknesses are exactly why we **validate against humans**.

2.12 Human evaluation and inter-annotator agreement

Even humans disagree a little. If two humans agree with a correlation of 0.80, we cannot expect the AI to do much better than that. So we also measure **human vs human** agreement and use it as a reference “ceiling”.

3. The maths we use (no fear!)

All metrics compare two lists of numbers: the judge's scores and the humans' average scores for the same 30 answers.

3.1 Pearson correlation (r)

Measures whether the two lists go up and down together in a **straight-line** way. Ranges from -1 to +1. +1 = perfect agreement, 0 = no relation, -1 = opposite.

3.2 Spearman rank correlation (ρ, “rho”)

Same idea, but first converts scores into **ranks** (1st best, 2nd best...). It only asks: *do the judge and humans put the answers in the same order?* This suits 1-5 ratings, so it is our **main metric**.

> **Tiny example:** Humans give answers A, B, C the scores 5, 3, 1. The judge gives 4, 3, 2. The numbers differ, but the order (A > B > C) is the same, so Spearman = 1.0 (perfect ranking agreement).

3.3 Cohen's kappa, quadratic weighted (κ)

Measures agreement on exact categories **after removing agreement that could happen by luck**. “Quadratic weighted” means being off by 1 point is punished a little and being off by 3 points is punished a lot. Since humans are averaged (e.g. 3.67), we round to the nearest whole number first.

3.4 MAE, within-1 and bias

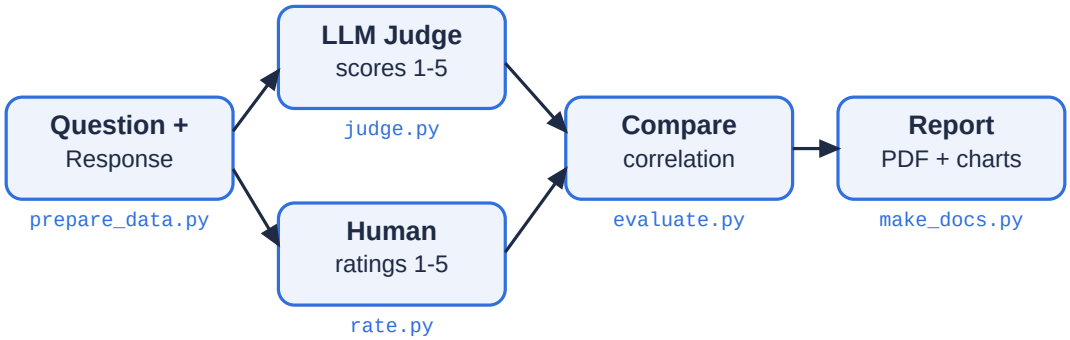
- **MAE (Mean Absolute Error):** average gap in points. MAE 0.5 = the judge is off by half a mark on average. Lower is better.
- **Within 1 point (%):** share of answers where judge and humans differ by at most 1 mark. Easy to explain in the viva.
- **Bias:** average of (judge - human). Positive = judge is more generous; negative = judge is stricter.

3.5 How to read the numbers

Correlation / kappa value	Meaning in plain words
0.80 - 1.00	Very strong agreement
0.60 - 0.79	Strong agreement
0.40 - 0.59	Moderate agreement
0.20 - 0.39	Weak agreement
below 0.20	Very weak / no agreement

(These bands are a common rule of thumb, not a strict law.)

4. How our pipeline works



- **Step 0 – Data:** `prepare_data.py` downloads HelpSteer2 and picks 120 samples; our own 30 samples are already in `data/samples.json`.
- **Step 1 – Human ratings:** HelpSteer2 already has them. For our 30 answers, each team member runs `rate.py` and scores them (1-5 for each criterion) *without seeing the judge's scores*.
- **Step 2 – LLM judge:** `judge.py` sends each pair with the rubric prompt to the LLM, reads the JSON reply and saves the scores and a one-line reason.
- **Step 3 – Compare:** `evaluate.py` averages the humans, computes all metrics per criterion, finds the biggest disagreements and draws two charts.
- **Step 4 – Report:** `make_docs.py` puts the real numbers and charts into Report.pdf.

5. The datasets and why we chose them

5.1 HelpSteer2 (main, 120 samples)

HelpSteer2 (Wang et al. 2024, NVIDIA) is a free public dataset of about 21,000 prompts with model answers. Each answer was rated by several trained human annotators on a 0-4 scale for **helpfulness**, **correctness**, coherence, complexity and verbosity. The released score is the annotators' average after removing outliers. So it gives us **real human ratings** without us rating anything.

How we picked the 120 samples (in `prepare_data.py`):

- Use the **validation split** (1,038 samples).
- Keep only **single-turn** conversations (one question, one answer) so the judge sees the full context.
- Keep only **short** samples (question + answer up to 1,500 characters) so we stay inside the free API limits.
- **Stratified sampling:** take 24 samples from each helpfulness level (0, 1, 2, 3, 4). Otherwise most samples would be good answers, and a judge that always says “4” would look great.
- Convert 0-4 to our 1-5 scale by adding 1. A fixed random seed (42) means anyone gets the same 120 samples.

5.2 Our own set (30 samples, for tone)

We wrote 10 questions (geography, Python, emotional support, science, health, biology, workplace email, computers, maths, literature). For each question there are **3 answers of different quality**:

Type of answer	Helpful?	Correct?	Polite?	Example (question: capital of Australia)
Good answer	Yes	Yes	Yes	“Canberra. Many guess Sydney, but...”

Polite but wrong	No	No	Yes	“The capital is Sydney...”
Correct but rude	Partly	Yes	No	“Seriously? It's Canberra. Look it up...”
Vague / too short	Low	Yes	Neutral	“RAM and ROM are both types of memory.”

Why? HelpSteer2 has no tone rating, and rude answers are rare in it. Also, if every answer were good or every answer bad, all three scores would move together and we could not tell whether the judge really separates the criteria. Mixing “correct but rude” and “polite but wrong” tests whether the judge scores *each criterion on its own*.

6. Code walk-through (file by file)

File	What it does in simple words
config.py	One place for all settings: team names, which LLM, API address, temperature, file names.
prepare_data.py	Downloads HelpSteer2, keeps short single-turn samples, picks 120 (24 per helpfulness level), saves data/helpsteer_samples.json and data/helpsteer_human.csv.
data/samples.json	Our own 30 questions and answers.
rate.py	Shows each answer in the terminal and asks a human for three scores (1-5). Saves to data/human_ratings.csv. If you stop midway, running it again continues where you left off.
judge.py	Reads the key from api_key.txt, builds the prompt, calls the LLM, extracts the JSON, keeps scores between 1 and 5, retries on errors, saves results/judge_custom.csv and results/judge_helpsteer.csv. Already-judged samples are skipped on re-run; if the daily limit is hit it tells you to switch key.
evaluate.py	For each dataset: averages human scores, computes Pearson, Spearman, kappa, MAE, within-1, bias and human-human agreement, saves metrics.csv, disagreements.csv and two charts.
make_docs.py	Builds Report.pdf (2 pages + references) and Team_Details.pdf using the real results.

6.1 The most important part: the judge prompt (from judge.py)

```
You are a strict but fair evaluator of AI assistant answers.
Rate the response on three criteria from 1 (worst) to 5 (best).
HELPFULNESS - 1 = useless, 3 = partly useful, 5 = complete and practical
CORRECTNESS - 1 = mainly wrong, 3 = partly correct, 5 = fully correct
TONE        - 1 = rude, 3 = neutral or cold, 5 = polite and appropriate
Judge each criterion separately. Do not reward length by itself.
QUESTION: ...   RESPONSE: ...
Reply with ONLY this JSON:
{"helpfulness": n, "correctness": n, "tone": n, "reason": "..."}
```

6.2 How the reply is read

```
match = re.search(r"\{.*\}", text, re.DOTALL)   # find the {...} part
data = json.loads(match.group(0))                  # turn text into a dictionary
score = min(5, max(1, int(round(float(data[c])))))  # force it into 1..5
```

Even if the model adds extra words around the JSON, the regular expression finds the `{...}` part.

6.3 The core of the evaluation (from evaluate.py)

```
human_avg = human.groupby("id")[C].mean()          # average of all raters
pearsonr(judge, human)          # linear agreement
spearmanr(judge, human)         # ranking agreement  (main metric)
cohen_kappa_score(judge, human.round(), weights="quadratic")
```

```
(judge - human).abs().mean()    # MAE
```

7. How to run the project

7.1 One-time setup

- Install Python 3.9 or newer.
- Open a terminal inside the project folder and run: `pip install -r requirements.txt`
- Get a free API key from **console.groq.com/keys** (sign in with Google, click “Create API Key”, copy it).
- Create a file `api_key.txt` in the project folder and paste only the key in it. **Do not** paste the key into config.py.
- Open `config.py` and type your names, roll numbers and professor's name.

7.2 Run the steps

```
# Step 0: download HelpSteer2 and pick 120 samples (once)
python prepare_data.py

# Step 1: every team member rates OUR 30 answers (about 15 min each)
python rate.py

# Step 2: the LLM judge (reads the key from api_key.txt, about 15 min)
python judge.py

# Step 3: compare judge with humans
python evaluate.py

# Step 4: build Report.pdf and Team_Details.pdf with the real results
python make_docs.py
```

7.3 If something goes wrong

Problem	Fix
“No API key found”	Make sure api_key.txt is in the project folder and contains only the key.
“Daily free limit used up”	Paste a teammate's Groq key into api_key.txt and run judge.py again.
prepare_data.py download fails	Open the link it prints in your browser and save the file into the data folder.
pip cannot find reportlab	Try python -m pip install reportlab or (Anaconda) conda install reportlab.
Rate limit / 429 error	The code waits and retries by itself. If it keeps happening, set SLEEP_SECONDS = 8 in config.py.
Model not found	Check the current model names on console.groq.com/docs/models and update MODEL in config.py.
No internet / no key	Use Option B (Ollama) in config.py: install Ollama, run ollama pull llama3.2.
evaluate.py says skipped	Run prepare_data.py, rate.py and judge.py first.

8. Our results and what they mean

The judge (**gpt-oss-120b**) scored all 150 answers. These are the final numbers from `results/metrics_all.csv`:

Dataset	Criterion	n	Spearman ρ	Kappa	MAE	Within 1 pt	Bias
HelpSteer2	Helpfulness	120	0.69	0.69	0.76	84%	+0.01
HelpSteer2	Correctness	120	0.67	0.67	0.80	79%	+0.08
Our set	Helpfulness	30	0.88	0.87	0.37	93%	+0.23
Our set	Correctness	30	0.89	0.85	0.43	90%	-0.03
Our set	Tone	30	0.85	0.88	0.37	93%	+0.03

8.1 How to explain these numbers

- **Strong agreement on real data.** On HelpSteer2 the judge ranks answers almost the same way the trained annotators do (ρ ≈ 0.7 = “strong” in the table in Section 3.5). Kappa says the same.
- **Usually within one mark.** For about 4 out of 5 HelpSteer2 answers, the judge's score is at most one point away from the human score. The average gap (MAE) is under one point.
- **No systematic bias.** The bias values are almost zero, so the judge is not simply more generous or stricter than humans.
- **Tone works too.** On our own set the judge separates tone from facts: it gave the rude-but-correct answers high correctness and low tone (ρ = 0.85 for tone).
- **Why is our set higher?** We wrote those answers to be clearly good or clearly bad, which is easier. HelpSteer2 answers are realistic and harder. Our set also has only one human rater (Tushar), so its numbers are less reliable than the HelpSteer2 ones.

8.2 Where the judge went wrong (good for the presentation)

- **Too generous:** “How do I make a Trident cocktail?” The answer gave a confident recipe. The judge gave 5/5 (“clear, accurate recipe”); the annotators gave 1/1 because the recipe is wrong. *Lesson:* the judge cannot fact-check things it does not know.
- **Too strict:** a prompt asking the assistant to tag new concepts with [concepts]. The judge gave 1 and 2; the annotators gave 4 and 5. The judge read the instruction differently.
- Among the 5 biggest disagreements, the judge was more generous in 2 and stricter in 3, so the errors go both ways.

One-line conclusion for the viva: “On 120 real human-labelled answers our LLM judge reached a Spearman correlation of about 0.7 with near-zero bias, so it is a reliable cheap substitute for human scoring of helpfulness and correctness, but it still cannot catch confident factual errors it doesn't know about.”

9. Submission checklist (deadline: 23 October 2026, 23:59 IST)

- **Done:** names and roll numbers, data, human ratings, judge, evaluation, Report.pdf, slides.
- Type the professor's name in config.py (PROFESSOR = ...) and run python make_docs.py once more.

- Use **Report.pdf inside llm_judge_project**, not the old Report_DRAFT.pdf.
- **Delete api_key.txt** from the folder.
- Upload to your IIITG Google Drive: the whole code folder (Source Code), Team_Details.pdf, Report.pdf (and this explanation if useful).
- Share the Drive link with the professor from your official IIITG email.
- Remember: each day late costs 10% of the 20 marks.

10. Presentation plan (7-8 minutes)

#	Slide	What to say	Time
1	Title & team	Project name, members.	0:20
2	The problem	LLM answers are free text; humans are slow; BLEU/ROUGE miss meaning and tone.	0:50
3	LLM-as-a-Judge idea	One AI grades another using a rubric; three criteria.	0:50
4	Pipeline diagram	Data → judge → human ratings → compare → report.	1:00
5	Datasets	HelpSteer2 (120, real human labels) + our 30 for tone.	0:50
6	Judge prompt	The rubric, gpt-oss-120b, temperature 0, JSON output.	0:50
7	Results	ρ 0.69 / 0.67 on HelpSteer2, 0.85 for tone; within 1 point for about 80%.	1:00
8	Chart	All bars above 0.6; why our set is higher.	0:40
9	Where the judge failed	Trident cocktail (too generous) and the [concepts] prompt (too strict).	0:50
10	Takeaways	Reliable and cheap, but cannot fact-check unknown facts; next steps. Questions.	0:40

Tip: a live demo of judge.py scoring one rude answer takes 30 seconds and impresses evaluators.

11. Likely viva questions with short answers

Q. Why not use BLEU or ROUGE?

They only count word overlap with a reference answer. They cannot tell if an answer is useful, true, or polite.

Q. Why use an LLM as a judge at all?

It is fast, cheap and can read meaning like a human, so it scales to thousands of answers.

Q. Why validate against humans?

LLM judges have biases (length, position, leniency). Human ratings are the gold standard, so agreement with them tells us if the judge can be trusted.

Q. Why Spearman and not only Pearson?

Our ratings are ordered categories (1-5). Spearman only checks ranking, so it suits this kind of data better.

Q. What is weighted kappa?

Agreement corrected for chance; bigger disagreements are punished more (quadratic weights).

Q. Why temperature 0?

So the judge is consistent and the results are reproducible.

Q. Why three separate criteria?

An answer can be correct but rude, or polite but wrong. One overall score would hide this.

Q. What if the judge's JSON is broken?

We find the {...} part with a regular expression, clip scores to 1-5, and retry up to 3 times.

Q. Why HelpSteer2?

It is public and recent, and every answer already has ratings from trained human annotators for helpfulness and correctness, so our validation uses real human labels, not only our own.

Q. Why only 120 HelpSteer2 samples?

The free Groq API has a daily token limit. 120 short samples fit in it, and 120 is enough for a reasonably stable correlation estimate.

Q. Why stratified sampling?

Most HelpSteer2 answers are good. Taking equal numbers from each score level tests the judge on bad answers too.

Q. Why your own 30 samples as well?

HelpSteer2 has no tone label, so we built a small set with rude and polite answers and rated tone ourselves.

Q. The scales differ (0-4 vs 1-5)?

We add 1 to HelpSteer2 scores. Correlations are not affected by this shift.

Q. What are the limitations?

Short single-turn samples only, tone checked on 30 answers rated by our team, one judge model and one prompt.

Q. How can it be improved?

Use the full HelpSteer2 set, several judge models, pairwise comparison, few-shot examples in the prompt.

Q. Pointwise vs pairwise?

Pointwise gives one answer a score; pairwise picks the better of two. We use pointwise because we need scores per criterion.

Q. Is the judge's answer key needed?

No, our judge is reference-free; it relies on its own knowledge.

12. Glossary

Word	Meaning
LLM	Large Language Model: a very large AI model trained on text.
Judge / evaluator	The LLM that gives marks to other answers.
Rubric	Marking scheme that says what each score means.
Prompt	The instruction text sent to the LLM.
Temperature	Setting for randomness; 0 means consistent.
API	A way for our code to talk to the LLM server.

JSON	Simple text format for structured data.
Gold standard	The most trusted reference; here, human ratings.
Correlation	How much two lists of numbers move together (-1 to +1).
Inter-annotator agreement	How much human raters agree with each other.
Validation	Checking a method against trusted results before trusting it.

Key papers (for further reading): Wang et al. 2024, *HelpSteer2* (arXiv:2406.08673); Zheng et al. 2023, *Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena* (NeurIPS); Liu et al. 2023, *G-Eval* (EMNLP); Chiang and Lee 2023, *Can Large Language Models Be an Alternative to Human Evaluations?* (ACL); Kim et al. 2024, *Prometheus* (ICLR).